

EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY



Đại Học Quốc Tế Miền Đông
Eastern International University

**Extending Network Service Simulation for
Malicious Package Analysis**

Bùi Bảo Minh – 2231209003
Đặng Thế Nhân - 2231200022

Software Engineering

Ho Chi Minh City, March, 2026

**EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY**



**Đại Học Quốc Tế Miền Đông
Eastern International University**

**Extending Network Service Simulation for
Malicious Package Analysis**

**Bùi Bảo Minh – 2231209003
Đặng Thế Nhân - 2231200022**

Software Engineering

Ho Chi Minh City, March, 2026

PACK-A-MAL PROJECT PROGRESS REPORT

Extending Network Service Simulation for Malicious Package Analysis

Item Information

Report type: Student progress report

Topic: Research and extension of Pack-a-mal for malicious package analysis

Member 1: Bui Bao Minh - 2231209003

Member 2: Dang The Nhan - 2231200022

Supervisor: [Vu Duc Ly]

Major: Software Engineering

Duration: 01/2026 – 03/2026

Location: Ho Chi Minh City

Ho Chi Minh City, March 2026

Contents

ABSTRACT	5
ACKNOWLEDGEMENT	5
1 INTRODUCTION	5
2 IMPLEMENTATION PROGRESS	5
2.1 Pack-a-mal architecture study	5
2.2 Docker container communication experiment	6
2.3 Building the Service Simulation module with INetSim	7
2.4 Integrating dead-URL checks and redirection to INetSim	7
2.5 Applying pURL and extending HTTP simulation	7
2.6 Designing three network operating modes	8
3 RESULTS ACHIEVED	8
4 LIMITATIONS AND FUTURE WORK	9
5 CONCLUSION	9
REFERENCES	9

ABSTRACT

This report presents the current progress of researching and extending the Pack-a-mal project to strengthen dynamic analysis through network service simulation. In this phase, the team focused on four main tasks: studying the Pack-a-mal architecture, validating communication between Docker containers, building a Service Simulation module with INetSim, and integrating a dead-URL checking mechanism to redirect traffic into a simulated environment.

In addition, the team experimented with Package URL (pURL) support, extended HTTP request analysis, and designed three network operating modes: Full mode, Half mode, and Transparent mode. The results show that this approach is feasible and provides a technical foundation for deeper integration into the Pack-a-mal worker in the next development stages.

ACKNOWLEDGEMENT

The team would like to sincerely thank the academic supervisor for providing technical guidance and valuable feedback throughout the project. We also thank all team members for collaborating in code review, Docker environment setup, network experiments, and result consolidation to complete this progress report.

1 INTRODUCTION

Malicious packages in open-source ecosystems can perform harmful actions immediately during installation or import. Therefore, dynamic analysis in a sandboxed environment is an important approach for detecting behaviors such as abnormal network connections, process creation, file manipulation, or remote payload retrieval.

Pack-a-mal is a system for analyzing suspicious packages. However, to better observe packages that attempt outbound communication, the system needs an additional network simulation layer. Based on that requirement, the team defined three main objectives: understand the Pack-a-mal architecture, build a network service simulation module, and test the integration of a dead-URL redirection mechanism into the analysis workflow.

2 IMPLEMENTATION PROGRESS

2.1 Pack-a-mal architecture study

Through source code inspection and environment setup, the team identified two main functional blocks in Pack-a-mal: the Dynamic Analysis Module and the Web APIs. The Dynamic Analysis Module is responsible for executing packages inside a sandbox and monitoring system calls, file operations, and network activity. The Web APIs provide a dashboard, REST endpoints, and access to analysis reports.

From this study phase, the team concluded that network simulation can act as a bridge between

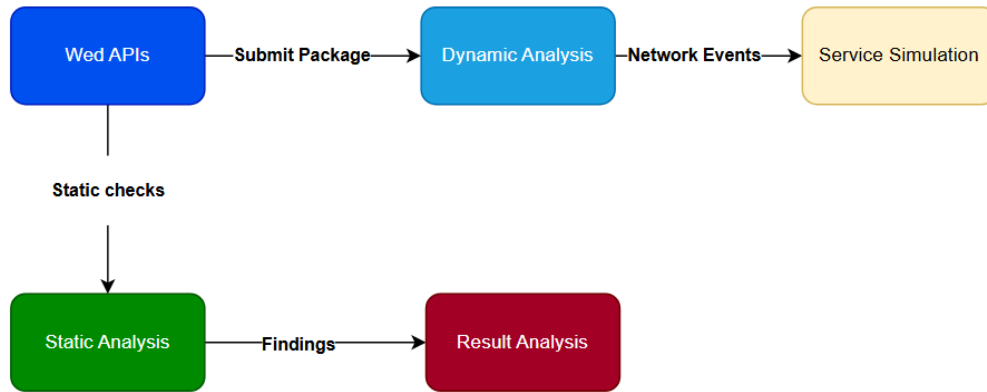


Figure 1: Pack-a-mal architecture overview.

the sandbox and the behavior-monitoring layer, especially for malware that connects to dead URLs or unavailable Internet services.

2.2 Docker container communication experiment

Before integrating the network module into the main system, the team built a small experiment with two containers. One container runs a Flask API as the server, while one Ubuntu container acts as the client. The team validated communication by testing ping, HTTP request/response, and Netcat-based TCP checks between the two containers.

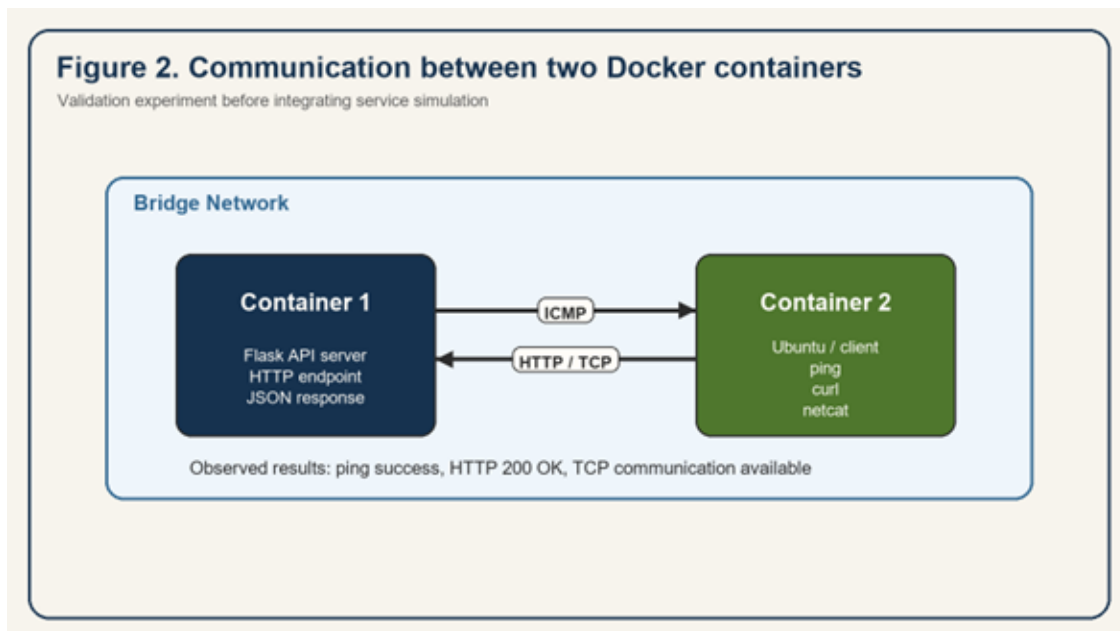


Figure 2: Communication between two Docker containers.

The experiment showed that all three scenarios were successful: ping worked at the network layer, HTTP returned 200 OK at the application layer, and Netcat verified that TCP ports were open for communication. This experiment served as the basis for moving on to the Service Simulation module.

2.3 Building the Service Simulation module with INetSim

The team designed a Docker Compose setup containing an INetSim container and a service simulation container. INetSim simulates DNS and HTTP services, while the service simulation container runs a Flask API for connectivity checks, configuration generation, and logging. The system is configured with the static IP 172.20.0.2 as the DNS nameserver, volume mounts for logs, and validation through curl and nslookup on localhost.

From the test cases, the team confirmed that both simulated DNS and simulated HTTP worked correctly. This is an important step because it allows the behavioral chain of a package to continue instead of failing immediately at the first unreachable request.

2.4 Integrating dead-URL checks and redirection to INetSim

The team created a sample package that attempts to connect to a non-existing URL, then added a URL liveness-checking module into the worker. If the URL is still alive, traffic follows the normal route. If the URL is dead, the system redirects the request to INetSim to generate a simulated response and continue observing package behavior.

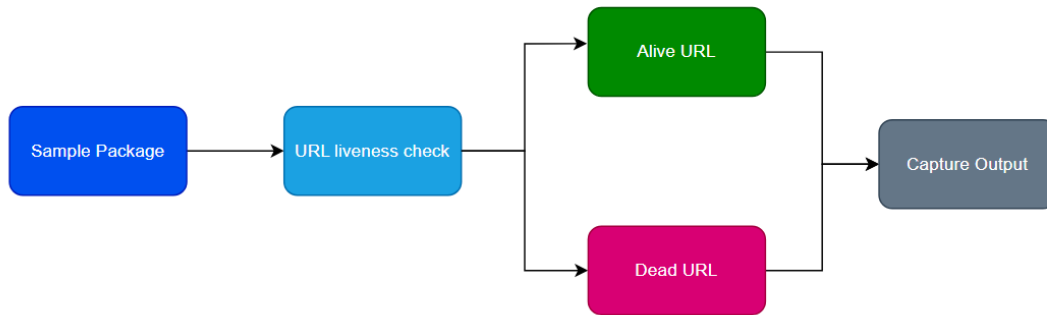


Figure 3: Redirect flow for dead URLs.

The test results show that dead URLs were redirected successfully and that the output logs captured the network behavior chain inside the simulated environment. This confirms that the proposed solution is suitable for safe dynamic analysis.

2.5 Applying pURL and extending HTTP simulation

The team modified relevant components to accept Package URL as an input format. This helps standardize the way analysis samples are submitted into the system. In parallel, the team developed an HTTP Request Analyzer, Request Classifier, Response Handler, and Safe Executable Handler to simulate context-aware HTTP responses and safely process executable download requests.

These extensions make the fake Internet environment more flexible and help the system identify suspicious requests such as executable downloads, data exfiltration, or command injection.

2.6 Designing three network operating modes

To support different analysis objectives, the team described and tested three network modes. Full mode redirects all traffic to INetSim to create a fully isolated environment. Half mode redirects only dead URLs, while alive URLs follow the normal route. Transparent mode does not modify traffic and only logs and analyzes metadata.

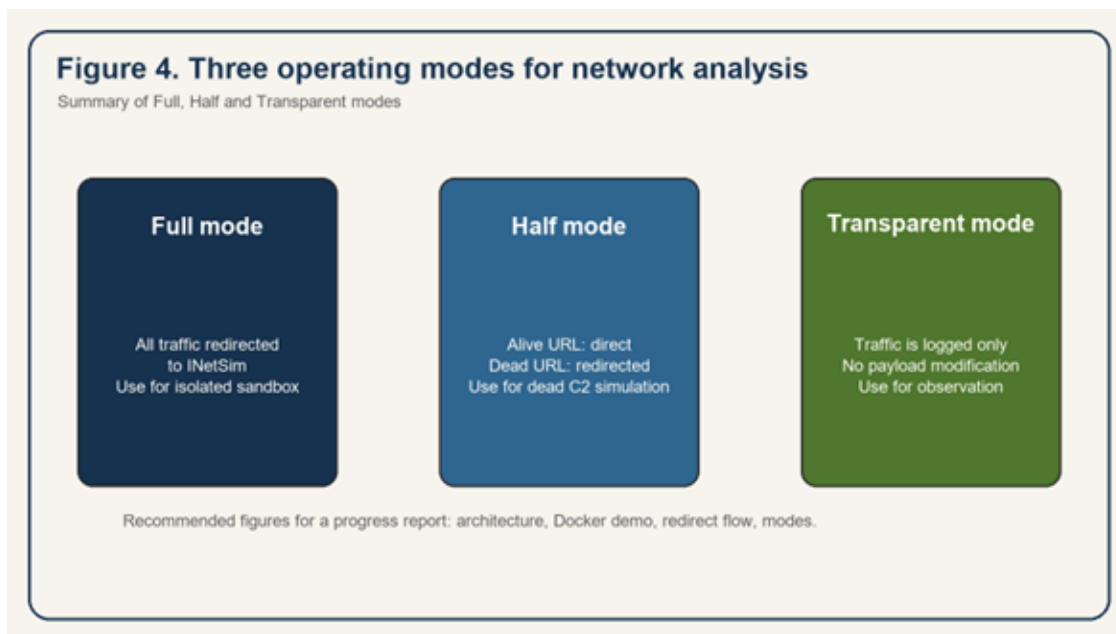


Figure 4: Three operating modes for network analysis.

These three modes allow the team to balance safety, traffic intervention, and observation value depending on the analysis scenario.

3 RESULTS ACHIEVED

At the current reporting stage, the team has achieved five main results. First, we understood the overall Pack-a-mal structure and ran the local environment successfully with Docker. Second, we completed a validation experiment proving stable communication between containers. Third, we implemented a Service Simulation module with INetSim for DNS and HTTP. Fourth, we integrated a dead-URL checking mechanism with redirection into the simulated environment. Fifth, we standardized input using pURL and made initial progress on extending HTTP request handling in a more intelligent way.

From a technical perspective, these results show that Pack-a-mal can be extended not only to monitor runtime behavior but also to actively provide simulated responses during malware analysis.

This is particularly useful for packages whose behavior depends on C2 infrastructure that is no longer active.

4 LIMITATIONS AND FUTURE WORK

The main limitation at this stage is that the service-simulation module is still at prototype level and has not yet been fully integrated into the production pipeline of Pack-a-mal. The cooperation between INetSim and FakeNet-NG remains at the design and local testing stage. In addition, the simulated responses are not yet diverse enough to cover more complex malware scenarios comprehensively.

In the next stage, the team plans to complete four main tasks. First, integrate network simulation more deeply into the worker and sandbox. Second, continue refining the cooperation model between INetSim and FakeNet-NG. Third, improve the quality of dynamic responses for HTTP, HTTPS, and executable downloads. Fourth, build additional sample packages to evaluate the system more systematically in terms of coverage and stability.

5 CONCLUSION

This progress report shows that extending Pack-a-mal with network service simulation is both feasible and practically valuable. The current results provide a clear enough foundation for the team to continue developing the system toward deeper behavior observation, more realistic network responses, and stronger support for dynamic analysis of malicious packages.

REFERENCES

1. Packguard Dev. Pack-a-mal. <https://github.com/packguard-dev/pack-a-mal>
2. INetSim. Internet Services Simulation Suite. <https://www.inetsim.org/>
3. Mandiant FLARE. FakeNet-NG. <https://github.com/mandiant/flare-fakenet-ng>
4. Seed Security Labs. <https://seedsecuritylabs.org/>
5. Internal technical document of the Pack-a-mal project team.